Customer Feedback System

ResolveFlow collects feedback from users on diagnosis quality and uses it to improve the eval suite and identify weak spots.

Architecture

```
User submits feedback
    ↓
/api/feedback/{thread_id} endpoint
    ↓
Stored in Postgres feedback table
    ↓
feedback_to_test_cases.py converts to eval cases
    ↓
capability_harness.py tests against them
    ↓
Regressions caught before deployment
```

API Endpoints

Submit Feedback

```
POST /api/feedback/{thread_id}
Authorization: Required (must be the thread owner)

Request body:
{
  "repo": "facebook/react",
  "issue_number": 12345,
  "verdict": "incorrect",  # "correct" | "incorrect" | "incomplete"
  "reason": "The diagnosis missed the root cause because..."
}

Response:
{
  "status": "feedback recorded"
}
```

Verdicts: - `correct` — Diagnosis was accurate and actionable - `incorrect` — Diagnosis was wrong or misleading - `incomplete` — Diagnosis missed important context or alternatives

Get Feedback Stats

```
GET /api/feedback/stats
No auth required

Response:
{
```

```
  "correct": 42,
  "incorrect": 3,
  "incomplete": 7,
  "total": 52
}
```

Workflow

1. Customer Uses ResolveFlow

User analyzes an issue, sees the diagnosis, and provides feedback:

```
Issue analyzed → Diagnosis shown → User rates verdict + reason
```

2. Feedback Collected in Database

```
SELECT * FROM feedback WHERE verdict = 'incorrect';

id | thread_id | repo | issue_number | verdict | reason
---|-----------|------|--------------|---------|-------
1  | abc123... | my/repo | 42 | incorrect | "Missed that this was a race condition"
2  | def456... | my/repo | 43 | incomplete | "Didn't mention the performance impact"
```

3. Convert to Test Cases

When you want to regenerate eval test cases from customer feedback:

```
python -m eval.feedback_to_test_cases
```

This: - Connects to Postgres - Fetches all "incorrect" and "incomplete" feedback - Generates `eval/customer_feedback_issues.py` with test cases - Grouped by frequency (most-reported issues first)

4. Run Evals

Add customer feedback issues to your capability harness:

```
# eval/capability_harness.py
from eval.customer_feedback_issues import CUSTOMER_FEEDBACK_ISSUES

def run_customer_feedback_tests():
    """Test against issues where users reported problems."""
    results = []
    for evidence in CUSTOMER_FEEDBACK_ISSUES:
        test = DiagnosisSeverityValidTest(evidence=evidence)
        result = test.run()
        results.append(result)
    return results

def main():
    all_results = (
        run_diagnosis_severity_tests() +
```

```
        run_diagnosis_citations_tests() +
        run_customer_feedback_tests()  # ← ADD THIS
    )
```

5. Monitor & Improve

Track over time:

```
Week 1: 5 "incorrect" feedback → Add to evals → All tests fail → Fix root cause
Week 2: 2 "incorrect" feedback → Evals catch regression immediately
Week 3: 0 "incorrect" feedback → System improving!
```

Best Practices

For Users

- **Be specific** in the reason: "Missed the async context" beats "wrong"
- **Verify first**: Make sure the feedback is actually incorrect before submitting
- **One issue at a time**: Report separate problems as separate feedback

For the Team

- **Review feedback weekly**: Check the `/api/feedback/stats` dashboard
- **Re-run eval-to-test-cases monthly**: Capture accumulated customer reports
- **Add to evals quarterly**: Periodically merge customer feedback issues into the eval suite
- **Close the loop**: When a feedback-driven fix is deployed, celebrate it (and measure if the verdict rate improves)

Example Flow

Day 1: Customer Reports Problem

```
POST /api/feedback/thread-123
{
  "repo": "my-company/my-app",
  "issue_number": 567,
  "verdict": "incorrect",
  "reason": "The diagnosis said 'memory leak' but it was actually a deadlock in the async handler"
}
```

Day 3: Generate Test Cases

```
$ python -m eval.feedback_to_test_cases
Found 1 issues with negative feedback:
  my-company/my-app#567: incorrect (1 reports)
    → The diagnosis said 'memory leak' but it was actually a deadlock...

✓ Generated eval/customer_feedback_issues.py
```

Day 5: Run Evals Against Feedback

```
$ python -m eval.capability_harness
✗ FAIL | diagnosis_severity_valid | my-company/my-app#567
        Expected: low/medium/high, got: "critical"

✗ FAIL | diagnosis_citations_no_hallucination | my-company/my-app#567
        Invalid citations: ["doc-99"]
```

Day 7: Fix the System

Investigate why the diagnosis failed on this issue. Usually: - The corpus doesn't have examples of this issue type → Add to ingest.py - The prompt isn't specific enough → Tune generate_diagnosis prompt - The retrieval is failing → Check MIN_RELEVANCE_SCORE in independent_review

Day 10: Verify Fix

Re-run feedback issues after deploying the fix — they should all pass now.

Metrics to Track

```
Dashboard should show:
- Feedback verdict distribution (% correct/incorrect/incomplete)
- Trending: Does % correct increase over time?
- Feedback volume: How many users are engaging?
- Issues with multiple reports: Which repos/patterns have problems?
```

Privacy & Security

- Feedback is only accessible to the user who submitted it (owned by `github_user_id`)
- The `/api/feedback/{thread_id}` endpoint requires auth and ownership check
- The `/api/feedback/stats` endpoint is public (aggregated stats only, no individual identities)
- No PII is stored; only repo/issue names and user's GitHub ID

Troubleshooting

No feedback showing up?

```
# Check if the feedback table has data
psql $DATABASE_URL -c "SELECT COUNT(*) FROM feedback;"

# See raw feedback
psql $DATABASE_URL -c "SELECT * FROM feedback LIMIT 10;"
```

Test cases not updating?

```
# Regenerate from scratch
python -m eval.feedback_to_test_cases

# Verify the file was created
cat eval/customer_feedback_issues.py
```

Stats endpoint showing zeros?

```
# Check database connectivity
python -c "from app.db import *; import asyncio; asyncio.run(get_feedback_summary(...))"
```